

Formal Verification of Imperative First-Class Functions in Move

FMCAD 2026

Wolfgang Grieskamp Teng Zhang Vineeth Kashyap Jake Silverman

Aptos Labs, Palo Alto, USA

- Large assets, irreversible transactions.
- Well-resourced adversaries: state actors took \$2B in 2025.

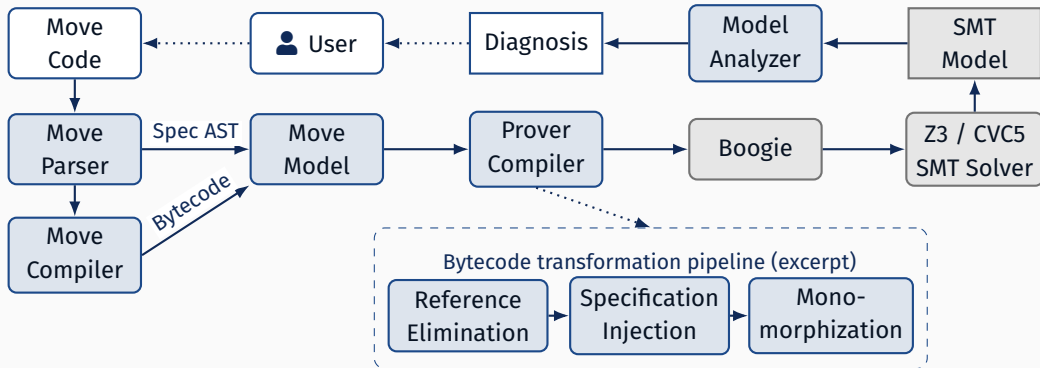
Billions per year lost to bugs and attacks

Stolen	2022	2023	2024	2025	H1 2026
	\$3.7B	\$1.7B	\$2.2B	\$3.4B	\$1.0–1.3B

Sources: Chainalysis, TRM Labs, CertiK

Blockchains are high assurance systems.

□ User □ Move Prover □ External (OSS)



- Specs are first-class citizens of code.
- Detailed diagnostics.
- Verifies bytecode after the compiler.
- Fast and reliable encoding (TACAS'22).

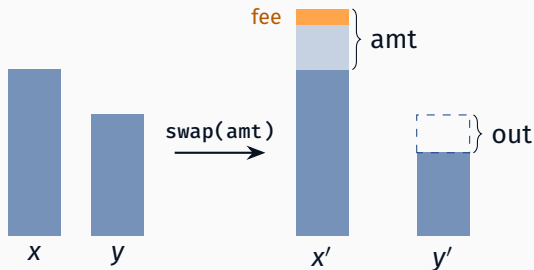
New concepts:

- Function types $T_1 \times \dots \times T_n \rightarrow T$: functions are first-class values.
- Closures capture values from their context, e.g. `delta` in

```
let f = |x| x + delta;
```

- A closure can be part of a struct and be persisted in global storage.
- Persisted function values are Move's principled account of **dynamic dispatch**.

Use cases: callbacks, extensible asset managers, dispatchable permissions, filter/map/reduce.



The trader pays amt of x and receives out of y , as priced by the curve; the fee stays in the pool. Constant product:

$$x \cdot y \leq x' \cdot y'.$$

```
// Liquidity pool of an automated market
// maker (AMM), as in Uniswap v2 (Ethereum)
struct Pool {
    reserve_x: u64, reserve_y: u64,
    // (reserve_in, reserve_out, amt) = out
    pricing: u64 × u64 × u64 → u64
}

// Extract of the AMM's core functionality
fun swap(pool: &mut Pool, amt: u64): u64 {
    let out = (pool.pricing)(
        pool.reserve_x, pool.reserve_y, amt);
    pool.reserve_x += amt;
    pool.reserve_y -= out;
    out
}
```

For a function value f with arguments $\bar{x}$ and results $\bar{y}$:

$\text{requires_of}\langle f \rangle(\bar{x})$	precondition holds
$\text{aborts_of}\langle f \rangle(\bar{x})$	the call aborts
$\text{ensures_of}\langle f \rangle(\bar{x}, \bar{y})$	postcondition holds (two-state)
$\text{result_of}\langle f \rangle(\bar{x})$	the result of the call (two-state)
$\text{reads_of}\langle f \rangle R$	memory the call may read
$\text{modifies_of}\langle f \rangle(\bar{x}) R[e_{\bar{x}}]$	memory the call may write ($R[e_{\bar{x}}]$: global resource)

- On a known function they unfold to its definition.
- On a parameter or a stored closure they stay abstract, constrained by specs.

Specs can name the state in which a predicate is evaluated:

$S \models p$	evaluate p in state S
$S.. \models q, ..S \models q$	S is the pre-/post-state of a two-state predicate
$\forall S \text{ in } *, \exists S \text{ in } *$	quantification over all states

```
fun followed_by(f: &mut A → (), g: &mut A → (), x: &mut A) {
  f(x); g(x)
} spec {
  ensures  $\exists S \text{ in } *: (S.. \models \text{ensures\_of}\langle f \rangle(x)) \ \&\& \ (S.. \models \text{ensures\_of}\langle g \rangle(x))$ 
}
```

The post-state of f is the pre-state of g , with no values threaded through the spec.

A data invariant on `Pool` constrains *any* closure stored in `pricing`:

1. never aborts,
2. output bounded by output reserve,
3. monotone in the input,
4. preserves the product of reserves.

Checked when a `Pool` is built, assumed wherever a `Pool` is used.

```
spec Pool { // Invariants of struct Pool
  reads_of<self.pricing> *;

  invariant  $\forall S \text{ in } *, ri: u64, ro: u64, a: u64:$ 
     $S \mid\sim !\text{aborts\_of}\langle\text{self.pricing}\rangle(ri, ro, a);$  (1)

  invariant  $\forall S \text{ in } *, ri: u64, ro: u64, a: u64:$ 
     $S.. \mid\sim \text{result\_of}\langle\text{self.pricing}\rangle(ri, ro, a) \leq ro;$  (2)

  invariant  $\forall S \text{ in } *, ri: u64, ro: u64, a1: u64, a2: u64:$ 
     $S.. \mid\sim a1 \leq a2 \implies \text{result\_of}\langle\text{self.pricing}\rangle(ri, ro, a1) \leq \text{result\_of}\langle\text{self.pricing}\rangle(ri, ro, a2);$  (3)

  invariant  $\forall S \text{ in } *, ri: u64, ro: u64, a: u64:$ 
     $S.. \mid\sim (ri + a) * (ro - \text{result\_of}\langle\text{self.pricing}\rangle(ri, ro, a)) \geq ri * ro;$  (4)
}
```

```
spec swap {  
  aborts_if pool.reserve_x + amt > MAX_U64;  
  ensures result = old(result_of<pool.pricing>(pool.reserve_x, pool.reserve_y, amt));  
  ensures pool.reserve_x * pool.reserve_y ≥ old(pool.reserve_x * pool.reserve_y);  
}
```

- pricing cannot abort by (1), reserve_y - out cannot underflow by (2): only the overflow abort remains.
- Product preservation follows from (4).

Problem reduced to first-order: the application side is no harder than first-order code.

Storing f where p is expected requires, for all inputs,

$$\text{requires_of}\langle p \rangle(\bar{x}) \Rightarrow \text{requires_of}\langle f \rangle(\bar{x})$$
$$\text{aborts_of}\langle f \rangle(\bar{x}) \Leftrightarrow \text{aborts_of}\langle p \rangle(\bar{x})$$
$$\text{ensures_of}\langle f \rangle(\bar{x}, \bar{y}) \Rightarrow \text{ensures_of}\langle p \rangle(\bar{x}, \bar{y})$$

Maybe hard to verify.

```
// fee-free constant product: verifies
fun product(ri: u64, ro: u64, a: u64): u64 { .. }

// with fee: abort invariant does not hold
struct Fee {bps: u64}
fun with_fee(owner: address,
             ri: u64, ro: u64, a: u64): u64 {
    let b = (a as u128)
        * (10000 - Fee[owner].bps as u128) / 10000;
    product(ri, ro, b as u64)
}
spec with_fee {
    // non-compliant aborts:
    aborts_if !exists<Fee>(owner);
    aborts_if Fee[owner].bps > 10000; ..
}
Pool{..,
    pricing: |ri, ro, a| with_fee(owner, ri, ro, a)}
```

After monomorphization this is known statically. Per function type τ , one datatype with a constructor per source:

datatype τ {
 $C_f(\overline{c} : \overline{T}), \dots$ closure of function f with captured values $\overline{c}$
 $P_{f,x}(), \dots$ function-typed parameter x of a verification target f
 $F_{\sigma.x}(n : \mathbb{N}), \dots$ struct field x of σ ; discriminator n separates locations
}

A VC only sees the function values derivable from the program.

```
proc  $invoke_{\tau}(x : \tau, \overline{p} : \overline{T})$  returns  $(\overline{r} : \overline{T})$   
  if  $x$  is  $C_f(\overline{c})$   
     $\overline{r} := \text{call } f(\overline{c}, \overline{p})$   
  else  
    assert  $requires_{\tau}(x, \mathcal{M}_{\tau}, \overline{p})$   
    if  $aborts_{\tau}(x, \mathcal{M}_{\tau}, \overline{p})$   
       $\text{abort\_flag} := \text{true}$   
    else  
      let  $\mathcal{M}'_{\tau} := \mathcal{M}_{\tau}$   
      havoc  $\Delta_{\tau}(\overline{p}), \overline{r}$   
      assume  $ensures_{\tau}(x, \mathcal{M}'_{\tau}, \mathcal{M}_{\tau}, \overline{p}, \overline{r})$ 
```

- Known closure: call the function.
- Otherwise: the usual opaque-call schema, driven by the behavioral predicates.
- $\mathcal{M}_{\tau}, \Delta_{\tau}$: read and write footprints of τ .

fun $aborts_{C_{foo}}[\mathcal{M}', \mathcal{M}](a)$
 $aborts_{\tau}(f, \mathcal{M}', a)$
 $\vee (\exists \mathcal{M}^S \bullet ensures_{\tau}(f, \mathcal{M}', \mathcal{M}^S, a)$
 $\wedge R[\mathcal{M}^S][a].0 > 0$
 $\wedge aborts_{\tau'}(g, \mathcal{M}^S, a))$

An abort at s needs s established first: the ensures reaching s are conjoined with the abort condition; pre-state aborts are disjointed in front.

Sound since a verified function satisfies

$$\bigvee \bar{A} \vee \bigwedge \bar{P}.$$

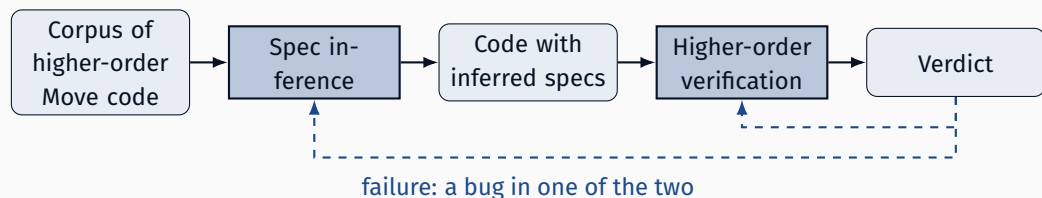
```

struct R(u64);
spec foo(f: address → (), g: address → (),
        a: address) {
  aborts_if aborts_of<f>(a);
  ensures ..S |~ ensures_of<f>(a);
  ensures S |~ R[a].0 > 0;
  aborts_if S |~ aborts_of<g>(a);
  ensures S.. |~ ensures_of<g>(a);
}

```

The state at s becomes an existentially quantified memory snapshot $\mathcal{M}^S$.

The Move Prover also has the reverse path: WP-based *specification inference* derives specs from code, using behavioral predicates for calls. Inferred specs are complete and are expected to verify.



Two independent algorithms are unlikely to agree on a bug. The current test suite covers 550 verification conditions.

- **Dafny**: behavioral predicates and labels, but function values cannot modify state, and labels are not quantified.
- **Verus, Prusti**: closure specs by state passing or separation logic; Move's alias-free references and resource-separated memory avoid both.
- **F***: effectful higher-order code with WP in the type system; heap separation is reasoned about in the logic, whereas Move separates memory at compile time.
- **Solidity verifiers** (Certora and others): dispatch by enumerating callees; our datatype makes the case split explicit and supports abstract function specs.

- Behavioral predicates and state labels make function specs first-class.
- The encoding discriminates on the variant reaching a call site: concrete effect when known, behavioral predicates otherwise.
- The Move Prover's routine, counterexample-driven verification now covers dynamic dispatch.

Future work

- Proof tactics for higher-order combinators (filter/map/reduce).
- Encoding and verification of the schema in Lean.

Thank you! Questions?

Prover, examples, and tests: github.com/aptos-labs/aptos-core

Paper examples: [third_party/move/move-prover/doc/higher-order-paper-26/examples](#)



Challenge: MVP is modular: a call site looks up the callee's spec. With dynamic dispatch there is no callee to look up. Solutions:

1. **Behavioral predicates:** $\text{requires_of}\langle f \rangle(\bar{x})$, $\text{aborts_of}\langle f \rangle(\bar{x})$, $\text{ensures_of}\langle f \rangle(\bar{x}, \bar{y})$ allow to reason about a function value f , making function specs first-class constructs.
2. **State labels:** name intermediate states, e.g.

$$\exists S \bullet ..S \mid \sim \text{ensures_of}\langle f \rangle(\bar{x}) \wedge S.. \mid \sim \text{ensures_of}\langle g \rangle(\bar{x})$$

3. **Encoding:** discriminate over the function values that can reach a call site.

```

fun find<T>(v: &vector<T>, pred: &T → bool): u64 {
  let i = 0; let n = v.length();
  while (i < n) {
    if (pred(v[i])) return i;
    i += 1;
  } spec {
    invariant i ≤ n;
    invariant no_match_before(v, pred, i);
  };
  n
}
spec find {
  ensures result ≤ len(v);
  ensures no_match_before(v, pred, result);
  ensures result < len(v)
    ⇒ result_of<pred>(v[result])
}

```

```

spec fun no_match_before<T>(v: vector<T>,
  pred: &T → bool, end: u64): bool
{ ∀ j in 0..end: !result_of<pred>(v[j]) }
fun find_zero_lambda(v: &vector<u64>): u64 {
  find(v, |x| x = 0
    spec { ensures result = (x = 0) })
}
spec find_zero_lambda {
  ensures result < len(v) ⇒ v[result] = 0
    && (∀ j in 0..result: v[j] ≠ 0);
  ensures result = len(v)
    ⇒ (∀ j in 0..len(v): v[j] ≠ 0);
}

```

The loop invariant is written once; callers verify from find's spec and the lambda's contract.

```
spec foo(f: address → (), g: T → S) {  
  reads_of<f> R;           // f may read resource R  
  reads_of<g> *;           // g may read anything in scope, plus unknown memory  
  modifies_of<f>(a: address) R[a]; // f may write R[a]; default is no writes  
}
```

- $\mathcal{M}_\tau = \mathcal{M}_{C_f} \cup \mathcal{M}_{P_{f,x}} \cup \mathcal{M}_{F_{\sigma,x}}$: derived from code for closures, declared via `reads_of` for parameters and fields.
- `reads_of<g> *` adds an abstract map for unknown memory, so $g(x)$ is not assumed equal across states.
- Δ_τ likewise, from `modifies` and `modifies_of`; no wildcard, since it would leave no frame.

$ensures_\tau$ switches over the variant and adds the frame condition (backup). It is a relation; Move functions are deterministic. Introduce an uninterpreted $result_\tau$ and tie it to $ensures_\tau$:

fun $result_\tau[\mathcal{M}', \mathcal{M}](x : \tau, \overline{p} : \overline{T}) \rightarrow \overline{U}$

axiom $\forall x, \overline{p} \bullet \{result_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p})\}$

let $\bar{r} := result_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p}) \bullet ensures_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p}, \bar{r})$

- One direction only: a biconditional would force all solutions of a weak *ensures* to coincide.
- For parameter and field variants, *ensures* is *defined* as the graph of an uninterpreted *result*.

The caller havoced all of Δ_τ ; the variant only constrains Δ_x :

```
fun ensures $_\tau$ [ $\mathcal{M}'$ ,  $\mathcal{M}$ ]( $x : \tau, \overline{p : T}, \overline{r : U}$ ) : bool
   $\bigwedge_{(\rho, a) \in \Delta_\tau}$  (
    if  $\rho \notin \mathcal{T}(\Delta_x)$ 
       $\mathcal{M}(\rho) = \mathcal{M}'(\rho)$ 
    else
      ( $\bigwedge_{(\rho, a') \in \Delta_x} a' \neq a$ )
       $\Rightarrow \mathcal{M}(\rho)[a] = \mathcal{M}'(\rho)[a]$ 
  )  $\wedge$  ensures $_x$ [ $\mathcal{M}'_x$ ,  $\mathcal{M}_x$ ]( $p, r$ )
```

Modification addresses are symbolic and may alias, hence the pairwise disequalities.

- Predicates as IVL functions *with bodies* time out: Boogie unfolds eagerly.
- Instead: *uninterpreted* predicates, connected by axioms with per-variant triggers.

axiom $\forall \bar{c}, \bar{p}, \bar{r} \bullet \{ \text{ensures}_\tau(C_f(\bar{c}), \bar{p}, \bar{r}) \}$
 $\text{ensures}_\tau(C_f(\bar{c}), \bar{p}, \bar{r}) \Leftrightarrow \text{ensures}_{C_f(\bar{c})}(\bar{p}, \bar{r})$

axiom $\forall \bar{p}, \bar{r} \bullet \{ \text{ensures}_\tau(P_{f,x}(), \bar{p}, \bar{r}) \}$
 $\text{ensures}_\tau(P_{f,x}(), \bar{p}, \bar{r}) \Leftrightarrow \text{ensures}_{P_{f,x}}(\bar{p}, \bar{r})$

axiom $\forall f, \bar{p}, \bar{r} \bullet \{ \text{ensures}_\tau(f, \bar{p}, \bar{r}) \}$
 $f \text{ is } F_{\sigma.x} \Rightarrow (\text{ensures}_\tau(f, \bar{p}, \bar{r}) \Leftrightarrow \text{ensures}_{F_{\sigma.x}}(f.n, \bar{p}, \bar{r}))$

Axioms fire only where the variant is syntactically present in the VC.

Category	Representative cases
Direct calls	Chained calls to a spec'd helper; inferred specs use <code>ensures_of</code> and <code>aborts_of</code> instead of inlining the body.
Mutual recursion	<code>is_even</code> and <code>is_odd</code> ; each spec is phrased via the other's behavioral predicate.
Higher-order parameters	Closure applied by the callee; spec abstracts via predicates on the parameter.
Stored-closure dispatch (enum)	Calculator with a closure-carrying enum variant.
Stored-closure dispatch (field)	Vault with a <code>Strategy</code> closure field on Aptos framework types; <code>modifies_of</code> inferred for the field.
Pre/post state labels	<code>move_to</code> , <code>move_from</code> , mutable resource indexing; <code>@pre</code> versus post-state memory.
Intermediate state labels	Sequenced state-modifying calls; existentially quantified labels pinned to defining predicates.

Developed in close collaboration with AI coding assistants (primarily Claude Code; Codex as a second opinion).

Went well

- Frontend extensions largely AI-produced, human review only.
- Root-cause diagnosis once inference surfaced a failing example.
- Timeout diagnosis via Boogie, SMT, and Z3 instantiation logs.

Went less well

- The encoding itself needed substantial human guidance; soundness calls were human.
- Dozens of iterations in the hard parts; effort approached manual implementation.
- Token cost.

Key enabler: the inference-then-verify pipeline gave an unambiguous correctness signal independent of human review.